

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Unit Tests This test file validates a customer order processing ETL pipeline. Here's a clear breakdown:---

****1. Test File Purpose****

Tests an AWS Glue job that:- Ingests customer and order data from S3- Cleans data (removes nulls, duplicates)- Implements SCD Type 2 (Slowly Changing Dimensions)- Generates customer aggregate spending reports- Registers tables in AWS Glue Data Catalog---

****2. Import Structure****

****Standard Libraries****

```
import sys, builtins
from unittest.mock import patch, MagicMock
from decimal import Decimal
from datetime import datetime, date
```

- `builtins`: Allows injecting Spark session globally (Glue pattern)- `unittest.mock`: Mocks S3 reads during testing- `Decimal`: Handles precise monetary values

****Test Framework****

```
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, ...
```

- `pytest`: Test runner with fixtures- PySpark types: Define schemas for test data

****Job Functions****

Imports all functions from `main.job` module to test them individually.---

****3. Test Fixtures (Reusable Test Data)****

****Fixture 1: Spark Session****

```
@pytest.fixture(scope="session")
def spark():
```

****Purpose****: Creates a local Spark session for all tests
****Key Steps****:1. Creates Spark with 2 local cores2. Injects into `\_\_builtins\_\_` (mimics Glue environment where `spark` is global)3. Yields session to tests4. Cleans up after all tests complete---

****Fixture 2: Sample Customer Data****

```
@pytest.fixture
def sample_customer_data(spark):
```

****Creates****: 5 clean customer records

```
C001 | John Doe | john@example.com | North
C002 | Jane Smith | jane@example.com | South
C003 | Bob Johnson | bob@example.com | East
C004 | Alice Brown | alice@example.com | West
C005 | Charlie Wilson | charlie@example.com | North
```

- Uses `get\_customer\_schema()` for correct structure- Converts column names to lowercase (matches job output)---

****Fixture 3: Customer Data with Nulls****

```
@pytest.fixturedef sample_customer_data_with_nulls(spark):
```

****Creates**:** Dirty data to test cleaning logic- Row 2: ``Null`` string in email (should be removed)- Row 4: Actual `None` in name field (should be removed)- Row 6: Duplicate of Row 2 (should be deduplicated)****Expected cleaning result**:** 3 rows removed → 3 valid rows remain---

****Fixture 4: Sample Order Data****

```
@pytest.fixturedef sample_order_data(spark):
```

****Creates**:** 5 clean order records

```
0001 | Laptop | $999.99 | Qty:1 | 2024-01-15 | C0010002 | Mouse | $25.50 | Qty:2 |  
2024-01-16 | C0010003 | Keyboard | $75.00 | Qty:1 | 2024-01-17 | C0020004 |  
Monitor | $299.99 | Qty:2 | 2024-01-18 | C0030005 | Headphones | $150.00 | Qty:1 |  
2024-01-19 | C004
```

****Important transformations**:-** Uses `Decimal` for prices (financial precision)- Converts date strings to PySpark `DateType`- Lowercases column names---

****Fixture 5: Order Data with Nulls****

```
@pytest.fixturedef sample_order_data_with_nulls(spark):
```

****Creates**:** Dirty order data- Row 3: ``Null`` string in itemname- Row 4: `None` in custid (orphaned order)- Row 6: Duplicate of Row 2---

****4. Test Cases****

****Test 1: Job Parameters****

```
def test_get_job_parameters():
```

****Validates**:** Configuration loading- Checks `customer\_source\_path` exists- Checks `order\_source\_path` exists- Verifies paths start with `s3://`---

****Test 2: Spark Initialization****

```
def test_initialize_spark_contexts(spark):
```

****Validates**:** Glue context setup- Returns SparkContext (`sc`)- Returns GlueContext (`glueContext`)- Returns SparkSession (`spark\_session`)- Returns Glue Job object (`job`)---

****Test 3: S3 Path Validation****

```
def test_validate_s3_access(spark):
```

****Tests**:-** Valid S3 path → Returns `True`- Local path → Returns `False`---

****Test 4-5: Schema Definitions****

```
def test_get_customer_schema():def test_get_order_schema():
```

****Validates****: Schema structure- Correct number of fields- Correct field names- Correct data types (e.g., `DecimalType` for prices)---

****Test 6: Customer Data Ingestion****

```
def test_ingest_customer_data(spark, sample_customer_data):
```

****Tests****: TR-INGEST-001 requirement****Steps****:1. Mocks `DataReader.read\_csv()` to return test data (avoids S3 access)2. Calls `ingest\_customer\_data()`3. Validates: - DataFrame is not None - Row count = 5 - Required columns exist (`custid`, `name`)---

****Test 7: Order Data Ingestion****

```
def test_ingest_order_data(spark, sample_order_data):
```

****Tests****: TR-INGEST-002 requirementSimilar to Test 6, but for orders:- Validates 5 rows- Checks `orderid`, `priceperunit` columns---

****Test 8: Customer Data Cleaning****

```
def test_clean_customer_data(spark, sample_customer_data_with_nulls):
```

****Tests****: TR-CLEAN-001 requirement****Validates cleaning logic****:1. Input: 6 rows (3 dirty)2. Output: < 6 rows (dirty rows removed)3. No `None` values remain4. No `""Null""` strings remain****Expected removals****:- Row with `None` in name- Row with `""Null""` in email- Duplicate row---

****Test 9: Order Data Cleaning****

```
def test_clean_order_data(spark, sample_order_data_with_nulls):
```

****Tests****: TR-CLEAN-002 requirement****Validates****:- Removes rows with null `custid`- Removes rows with `""Null""` strings- Removes duplicates- Output row count < input row count---

****5. Testing Strategy****

****Unit Testing Approach****

- ****Isolation****: Each function tested independently- ****Mocking****: S3 reads mocked to avoid AWS dependencies- ****Fixtures****: Reusable test data reduces duplication

****Coverage Areas****

1. ****Configuration****: Parameter loading2. ****Initialization****: Spark/Glue context setup3. ****Validation****: S3 path checks4. ****Schema****: Data structure definitions5. ****Ingestion****: CSV reading (mocked)6. ****Cleaning****: Null/duplicate removal logic

****Test Data Design****

- ****Clean data****: Validates happy path- ****Dirty data****: Validates error handling and cleaning logic- ****Edge cases****: Nulls, duplicates, string `""Null""` values---

****6. Key Testing Patterns****

****Pattern 1: Fixture Injection****

```
def test_clean_customer_data(spark, sample_customer_data_with_nulls):
```

Pytest automatically provides fixtures as function arguments.

****Pattern 2: Mocking External Dependencies****

```
with patch.object(DataReader, 'read_csv', return_value=sample_customer_data):
```

Replaces S3 reads with in-memory test data.

****Pattern 3: Assertion Chaining****

```
assert result_df is not Noneassert result_df.count() == 5assert 'custid' in result_df.columns
```

Multiple validations ensure comprehensive coverage.---

****Summary****

This test suite ensures the ETL job correctly:

1. ■ Loads configuration
2. ■ Initializes Spark/Glue contexts
3. ■ Validates S3 paths
4. ■ Defines correct schemas
5. ■ Ingests data from S3 (mocked)
6. ■ Cleans data (removes nulls, duplicates, invalid values)

****Note**:** The code snippet is incomplete (cuts off mid-assertion), but the pattern is clear for remaining tests.